Mohammad Fauzan P. D.

264107020159

TI – 1C / 19

Percobaan 1 :

Pertanyaan

1. Jika inisialisasi `i = 1` diubah menjadi `i = 0` tanpa mengubah kondisi, berapa karakter `*` yang tercetak ketika n = 5? Jelaskan.
2. Jika kondisi `i <= n` diganti menjadi `i < n`, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?
3. Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya.

```
for (int i = 2; i <= n; i += 2) {
    System.out.print(i + " ");
}
```

   Modifikasi kode tersebut agar mencetak bilangan **ganjil** positif yang tidak melebihi n.
4. Jelaskan mengapa perubahan *increment* `i++` menjadi `i--` (dengan kondisi tetap `i <= n`) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan **Ctrl + C** pada terminal.

Jawaban

1. **Jumlah karakter * yang tercetak: 6 karakter** (******)
   Penjelasan: Jika inisialisasi diubah menjadi int i = 0 dengan kondisi i <= n dan $n = 5$, maka nilai i yang memenuhi perulangan adalah 0,1,2,3,4,5. Total ada 6 iterasi, sehingga karakter * yang tercetak sebanyak 6 buah.

```
Masukkan Nilai n : 5
******
```

2. **Jumlah perulangan saat $n = 5$: 4 kali** perulangan.

   Mengapa hasilnya berbeda? Operator < berarti "kurang dari", sehingga nilai n (5) tidak ikut disertakan dalam kondisi. Nilai i yang memenuhi hanya 1,2,3,4. Akibatnya, jumlah iterasi berkurang 1 dibandingkan menggunakan operator <=.

3. Genap Positif

```
for (int i = 2; i <= n; i+= 2) {
    System.out.print(i + " ");
}
```

```
Masukkan Nilai n : 9
2 4 6 8
```

   Ganjil Positif

```
for (int i = 1; i <= n; i+= 2) {
    System.out.print(i + " ");
}
```

```
Masukkan Nilai n : 9
1 3 5 7 9
```

4. **Penjelasan mengapa i-- menyebabkan perulangan terus-menerus (*infinite loop*):** Jika inisialisasi dimulai dari angka positif (misal $i = 1$) lalu menggunakan *decrement* i--, nilai i akan **terus berkurang** (menjadi $0, -1, -2, -3, ...$). Karena n adalah bilangan positif dan nilai i justru semakin kecil, kondisi i <= n akan **selamanya bernilai benar (true)**. Akibatnya, program terjebak dalam *infinite loop* dan tidak akan pernah berhenti secara otomatis.

Percobaan 2 :

1. Tentukan fungsi variabel `i` dan `j` pada program tersebut.
2. Berapa kali *outer loop*, *inner loop*, dan perintah `print("*")` dijalankan untuk baris = 4 dan kolom = 7?
3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan `kolom`, inner menggunakan `baris`)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan.
4. Jelaskan akibat penghapusan `System.out.println();` setelah *inner loop*.
5. Modifikasi program agar karakter yang dicetak adalah `#`, lalu *commit* dan *push* perubahan ke GitHub.

Jawaban

1. **Variabel i**: Berfungsi sebagai penghitung/pengontrol untuk **perulangan luar (*outer loop*)** yang mengatur jumlah **baris** ke bawah.

   Variabel j: Berfungsi sebagai penghitung/pengontrol untuk **perulangan dalam (*inner loop*)** yang mengatur jumlah **kolom** atau karakter * yang dicetak menyamping pada setiap baris.

2. ***Outer loop* (Perulangan Luar)**: Dijalankan sebanyak **4 kali** (sesuai jumlah baris).

   ***Inner loop* (Perulangan Dalam)**: Dijalankan sebanyak **4 kali** (kepala perulangan dalam terpanggil 4 kali oleh *outer loop*).

   Perintah print("*"): Dijalankan sebanyak baris $\times$ kolom $= 4 \times 7 =$ **28 kali**.

3. **Apa yang terjadi?** Jika batas perulangan ditukar dengan menggunakan kolom sebagai batas *outer loop* dan baris sebagai batas *inner loop*, maka struktur perulangan akan berbalik.

   Penjelasan (menggunakan baris = 3 dan kolom = 6):

   Perulangan luar akan berjalan sebanyak **6 kali** (mengikuti nilai kolom).

   Perulangan dalam akan mencetak sebanyak **3 bintang** per baris (mengikuti nilai baris).

   Hasil cetak di layar akan berbentuk persegi panjang vertikal dengan **6 baris ke bawah dan 3 kolom ke samping** (kebalikan dari sebelumnya yang 3 baris dan 6 kolom).

4. Jika perintah System.out.println(); setelah *inner loop* dihapus, **semua karakter dari setiap baris akan tercetak menyambung menjadi satu baris panjang** tanpa ada perpindahan baris (enter).

   Contohnya untuk baris = 3 dan kolom = 6, alih-alih membentuk 3 baris ke bawah, seluruh 18 bintang akan berderet menyamping menjadi: **********.

5.
```
for (int i = 1; i <= baris; i++) {
    for (int j = 1; j <= kolom; j++) {
        System.out.print("#");
    }
    System.out.println();
```

```
Masukkan Jumlah Baris : 5
Masukkan Jumlah Kolom : 6
######
######
######
######
######
```

Percobaan 3 :

1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.
2. Jelaskan peran variabel `i` dan `j`. Mengapa nilai `j` harus diinisialisasi kembali ke 0 pada setiap iterasi *outer loop*? Apa yang terjadi jika `j` tidak di-*reset*?
3. Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (*update*) pada *outer loop* yang diperlukan untuk membentuk segitiga terbalik.

Jawaban

1. **Ketidaksesuaian Output Awal dan Perbaikan**

   - **Ketidaksesuaian Output:**

     Output awal menghasilkan bentuk gabungan dua segitiga (naik lalu turun seperti belah ketupat) karena program memiliki dua blok perulangan *while* secara berurutan. Selain itu, jumlah baris maksimal mencapai $n + 1$ (misal saat $n = 6$, tercetak hingga 7 baris bintang *******) karena inisialisasi dimulai dari i = 0 hingga i = n.

   - **Minimal Dua Perbaikan yang Diperlukan:**

     Menghapus blok *while* kedua yang mengatur pola turun, sehingga program hanya berfokus mencetak satu segitiga siku-siku.

     Menyesuaikan inisialisasi dan kondisi pada perulangan pertama (misalnya mengubah int i = 1 dan kondisi i <= n) agar jumlah baris maksimal sesuai persis dengan nilai n yang diinputkan pengguna.

2. **Peran Variabel i dan j, serta Pentingnya Me-*reset* j**
   - **Peran Variabel:**

     a. **Variabel i**: Berfungsi sebagai pengontrol *outer loop* (perulangan luar) untuk melacak dan menentukan baris yang sedang dikerjakan.
     b. **Variabel j**: Berfungsi sebagai pengontrol *inner loop* (perulangan dalam) untuk menghitung jumlah karakter * yang dicetak menyamping pada baris tersebut.

   - **Mengapa j harus diinisialisasi kembali ke 0 pada setiap iterasi *outer loop*?** Nilai j harus dikembalikan ke 0 di awal setiap baris baru agar penghitungan jumlah bintang selalu dimulai dari awal untuk setiap baris yang dicetak.
   - **Apa yang terjadi jika j tidak di-*reset*?** Jika j tidak di-*reset* (misalnya dideklarasikan di luar *outer loop*), nilainya akan terus bertambah dari baris-baris sebelumnya. Akibatnya, perulangan dalam tidak akan berjalan dengan benar atau gagal mencetak pola segitiga karena nilai j sudah melewati batas kondisi.

3. Untuk mengubah *outer loop* agar dapat membentuk pola segitiga terbalik (dari jumlah bintang terbanyak n turun hingga 1), diperlukan tiga perubahan utama berikut:

   - **Inisialisasi:** Dimulai dari nilai maksimum, yaitu n (contoh: int i = n;).
   - **Kondisi:** Diubah agar perulangan terus berjalan selama nilai pengontrol masih bernilai positif atau sesuai batas bawah (contoh: i >= 1; atau i > 0;).
   - **Perubahan Nilai (*Update*):** Menggunakan pengurangan atau *decrement* (i--;) agar pada setiap iterasi berikutnya jumlah baris/bintang semakin berkurang.

Percobaan 4 :

1. Mengapa `totalNilai` perlu di-*reset* di dalam *outer loop*, tetapi `rataTertinggi` tidak boleh di-*reset* pada setiap kelompok?
2. Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?
3. Apa yang terjadi jika `totalNilai` dideklarasikan sebagai `int` dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.
4. Mengapa `rataTertinggi` diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.
5. Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya.

Jawaban

1. **`totalNilai` perlu di-reset di dalam *outer loop*** karena variabel ini digunakan untuk mengakumulasi total nilai penilai **khusus untuk satu kelompok saja**. Jika tidak di-reset ke `0` untuk setiap kelompok baru, maka nilai total kelompok sebelumnya akan ikut terjumlah ke kelompok berikutnya, sehingga perhitungan rata-rata kelompok selanjutnya akan menjadi salah.

`rataTertinggi` tidak boleh di-reset pada setiap kelompok karena variabel ini berfungsi untuk **menyimpan rekor nilai rata-rata tertinggi secara keseluruhan** dari semua kelompok yang sudah diperiksa. Jika di-reset pada setiap kelompok, nilai rekor tertinggi sebelumnya akan hilang dan program tidak bisa membandingkan kelompok saat ini dengan kelompok terbaik sebelumnya.

2. Jika terdapat **3 kelompok** (`jumlahKelompok = 3`) dan **2 penilai** (`jumlahPenilai = 2`):

   Outer loop (`while`) berjalan sebanyak **3 kali** (untuk setiap kelompok).

   Di dalam setiap iterasi *outer loop*, *inner loop* (`for`) berjalan sebanyak **2 kali** (untuk setiap penilai).

   Total eksekusi badan perulangan FOR:

   Jumlah Kelompok × Jumlah Penilai $= 3 \times 2 = \mathbf{6}$ kali

3. **Apa yang terjadi:** Jika `totalNilai` bertipe `int` dan dibagi dengan `jumlahPenilai` yang juga bertipe `int` (misal menggunakan pembagian integer Java), maka komponen pecahan desimal dari hasil bagi akan terbuang (dibulatkan ke bawah ke bilangan bulat terdekat), meskipun variabel penampungnya (`rataNilai`) bertipe `double`.

   Contoh Nilai: Jika total nilai kelompok adalah `5` dan jumlah penilai adalah `2`, maka operasi `5 / 2` dalam integer akan menghasilkan `2`, bukan `2.5`. Hasil akhirnya akan tercatat sebagai `2.0` karena langsung dikonversi ke `double` setelah pembagian bulat terjadi. *(Catatan: Pada kode Anda, `totalNilai` sudah dideklarasikan sebagai `double`, sehingga pembagian desimal berjalan dengan akurat).*

4. **Mengapa -1:** Karena dalam sistem penilaian ini, nilai minimum yang mungkin didapat adalah 0 (atau lebih besar). Dengan memberikan nilai awal -1, dipastikan bahwa rata-rata dari kelompok pertama (berapapun nilainya, bahkan jika nilainya 0) pasti akan lebih besar dari -1, sehingga kelompok pertama otomatis akan tercatat sebagai rataTertinggi sementara.

   Contoh input yang membuat hasil salah jika nilai awalnya 0: Jika semua kelompok mendapatkan rata-rata di bawah 0 (atau jika rentang nilai memungkinkan negatif, meskipun di kode dibatasi 0–100), atau jika seluruh kelompok mendapatkan rata-rata persis 0, maka jika nilai awal 0, pembanding rataNilai > rataTertinggi ($0 > 0$) bernilai false, sehingga kelompok terbaik tidak akan pernah terperbarui dengan benar.

5. **Perulangan Luar (while - nomorKelompok)**:

   Jenis: while loop

   Peran: Mengatur perulangan proses penilaian untuk setiap **kelompok** dari kelompok pertama hingga kelompok terakhir (jumlahKelompok).

   Perulangan Tengah (for - j):

   Jenis: for loop

   Peran: Mengatur perulangan pengambilan masukan (input) nilai dari setiap **penilai** (jumlahPenilai) di dalam kelompok yang sedang aktif.

   Perulangan Dalam (do-while - validasi input):

   Jenis: do-while loop

   Peran: Melakukan **validasi input** agar program terus meminta ulang masukan nilai dari penilai jika pengguna menginput angka di luar rentang yang valid (yaitu harus antara 0 sampai 100).

Tugas 1 :

```java
import java.util.Scanner;

public class Tugas01_19 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int total = 0;
        System.out.print("Masukkan Nilai N : ");
        int n = sc.nextInt();

        for (int i = 0; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
            total += i;
            }

            System.out.println("Total Angka Yang Tercetak : " + total);
            System.out.println("");

            sc.close();
        }
    }
```

```
Masukkan Nilai N : 6

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
Total Angka Yang Tercetak : 21
```

Tugas 2 :

```java
import java.util.Scanner;

public class Tugas02_19 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int total = 0;
        System.out.print("Masukkan Nilai N : ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                System.out.print(i * j + " ");
                total += i * j;
            }
            System.out.println();
            total += i;
            }

            System.out.println("Jumlah Seluruh Angka : " + total);
            System.out.println("");

            sc.close();
    }
}
```

```
Masukkan Nilai N : 5
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
Jumlah Seluruh Angka : 240
```

Tugas 3 :



```java
import java.util.Scanner;

public class Tugas03_19 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int jumlahToko;
        do {
            System.out.print("Masukkan jumlah toko: ");
            jumlahToko = input.nextInt();
            if (jumlahToko < 1) {
                System.out.println("Jumlah toko minimal 1!");
            }
        } while (jumlahToko < 1);

        int totalSeluruh = 0;
        int barangTerbanyak = -1;
        int tokoTerbanyak = 0;

        for (int toko = 1; toko <= jumlahToko; toko++) {
            System.out.println();
            int jumlahPelanggan;
            do {
                System.out.print("Jumlah pelanggan toko " + toko + ": ");
                jumlahPelanggan = input.nextInt();
                if (jumlahPelanggan < 0) {
                    System.out.println("Tidak boleh negatif!");
                }
            } while (jumlahPelanggan < 0);

            int totalBarangToko = 0;

            for (int pelanggan = 1; pelanggan <= jumlahPelanggan; pelanggan++) {
                int barang;
                do {
                    System.out.print("  Jumlah barang pelanggan " + pelanggan + ": ");
                    barang = input.nextInt();
                    if (barang < 0) {
                        System.out.println("  Tidak boleh negatif!");
                    }
                } while (barang < 0);

                totalBarangToko += barang;
            }

            System.out.println("Toko " + toko + " - pelanggan: " + jumlahPelanggan
                    + ", total barang: " + totalBarangToko);

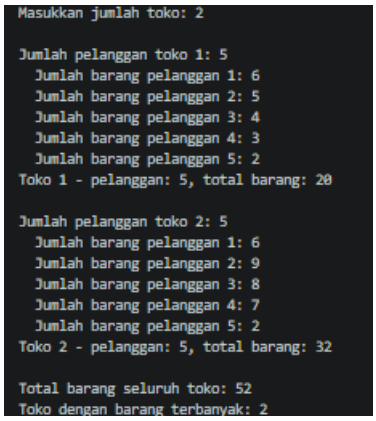
            totalSeluruh += totalBarangToko;

            if (totalBarangToko > barangTerbanyak) {
                barangTerbanyak = totalBarangToko;
                tokoTerbanyak = toko;
            }
        }

        System.out.println();
        System.out.println("Total barang seluruh toko: " + totalSeluruh);
        System.out.println("Toko dengan barang terbanyak: " + tokoTerbanyak);

        input.close();
    }
}

```



```
Masukkan jumlah toko: 2

Jumlah pelanggan toko 1: 5
  Jumlah barang pelanggan 1: 6
  Jumlah barang pelanggan 2: 5
  Jumlah barang pelanggan 3: 4
  Jumlah barang pelanggan 4: 3
  Jumlah barang pelanggan 5: 2
Toko 1 - pelanggan: 5, total barang: 20

Jumlah pelanggan toko 2: 5
  Jumlah barang pelanggan 1: 6
  Jumlah barang pelanggan 2: 9
  Jumlah barang pelanggan 3: 8
  Jumlah barang pelanggan 4: 7
  Jumlah barang pelanggan 5: 2
Toko 2 - pelanggan: 5, total barang: 32

Total barang seluruh toko: 52
Toko dengan barang terbanyak: 2
```